

1b.

$$\frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \begin{bmatrix} 1 \\ t_i^2 (1 - t_i^2) \\ e^{t_i} \end{bmatrix}$$

1c. This gradient can be applied opposite to the parameters, slowly driving them to have less loss (gradient descent).

```
3.
[
    [1,0,0,0,0],
    [0,1,0,0,0],
    [0,0,1,0,0],
    [0,0,1,0,0],
    [0,0,0,1,0],
    [1,0,0,0,0],
    [0,0,0,0,1],
    [0,1,0,0,0]
]
```

9a. My Image

```
# 10.
class NN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(300, 200)
        self.fc2 = nn.Linear(200, 1000)
        self.fc3 = nn.Linear(1000, 50)

    def forward(self, x):
        x = torch.sigmoid(self.fc1(x)) #
        activation function
        x = torch.sigmoid(self.fc2(x)) #
        activation function
        x = torch.sigmoid(self.fc3(x)) #
        activation function
        return x

#
-----
----

model = NN().to(device)

ds = TensorDataset(X_train_tensor, y_train)
loader = DataLoader(ds, batch_size=64, shuffle=True)
```

```

criterion = nn.MSELoss()                                # gradient
descent definition
optimizer = torch.optim.SGD(model.parameters(), lr=0.5) # gradient
descent definition

model.train()

for epoch in range(10):
    for x, y in loader:
        x, y = x.to(device), y.to(device)

        p = model(x)                                     # forward
method application

        loss = criterion(p, y)
        optimizer.zero_grad()
        loss.backward()                                   # backprop
application
        optimizer.step()                                 # gradient
descent application

```

11a. $\sigma = \frac{1}{1+e^{-x}}$

```

# 13
def fit(self, X, c):
    self.losses = []
    for iteration in range(self.iters):
        probability = self.predict_probability(X)          #
forward
        p_c = np.array((probability - c)).reshape((len(c), 1)) #
forward
        gradient = np.dot(X.T, (probability - c).ravel())  #
backward
        self.w -= self.lr * gradient                       #
backward
        self.losses.append(self.loss(probability, c))

```

1. Chain Rule